

Tecnológico de Monterrey

Auditoría y "Parcheo" Algorítmico

El Enrutador Defectuoso

Franco De Escondrillas Vázquez | A01739410

Octavio Hernández Loyo | A01739304

Ingeniería en Tecnologías Computacionales

Dr. Sandra Eugenia Barajas Montiel

TC2038

8 de octubre, 2026

1. Análisis de errores

Se encontraron dos errores en la primera versión del código (*bellman_buggy*) siendo uno más grave que el otro. En la declaración de la variable de *contadorVertices* simplemente se replicaba el valor de numeroVertices para después ocuparlo en el ciclo de asignación de pesos a las aristas, sin embargo al hacer esto no se toma en cuenta que el número de iteraciones de relajación es el número de vértices menos uno. En nuestro código corregido se retira por completo la variable de *contadorVertices* y solo se ocupa el valor de *numeroVertices - 1* para el ciclo.

El otro error, que consideramos más que un error algo incompleto, es que en la declaración de la función de *ejecutarBellmanFord* se declara su valor de retorno como un booleano. Fuera del ciclo principal existe un *return False* en caso de no encontrar ciclos negativos, pero para cuando se encuentra algún ciclo negativo no existe ninguna revisión de los valores para encontrar el ciclo y por lo tanto ningún *return True* que nos indique que lo encontró. Se agregó para la búsqueda de ciclos negativos un ciclo donde se recorre las aristas en su lista haciendo el calculo con su peso y el destino, si se encuentra un valor menor después de las iteraciones de relajación significa que puede seguir disminuyendo el valor de las aristas es decir que existe un ciclo negativo en el grafo.

Ahora que el main.cpp puede recibir un valor booleano true del algoritmo, se puede mostrar el mensaje de error en caso de encontrar el ciclo negativo del grafo.

2. Traza caso personalizado

El grafo que se muestra en la figura 1 cuenta con un ciclo negativo en las aristas [1 › 5 › 4], el cual agrega un valor de – 1 cada vez que se recorre. Si dicho ciclo se omitiera, el camino más corto de 0 a 6 tendría un valor de 1, transitando por la ruta [0 › 2 › 3 › 6].

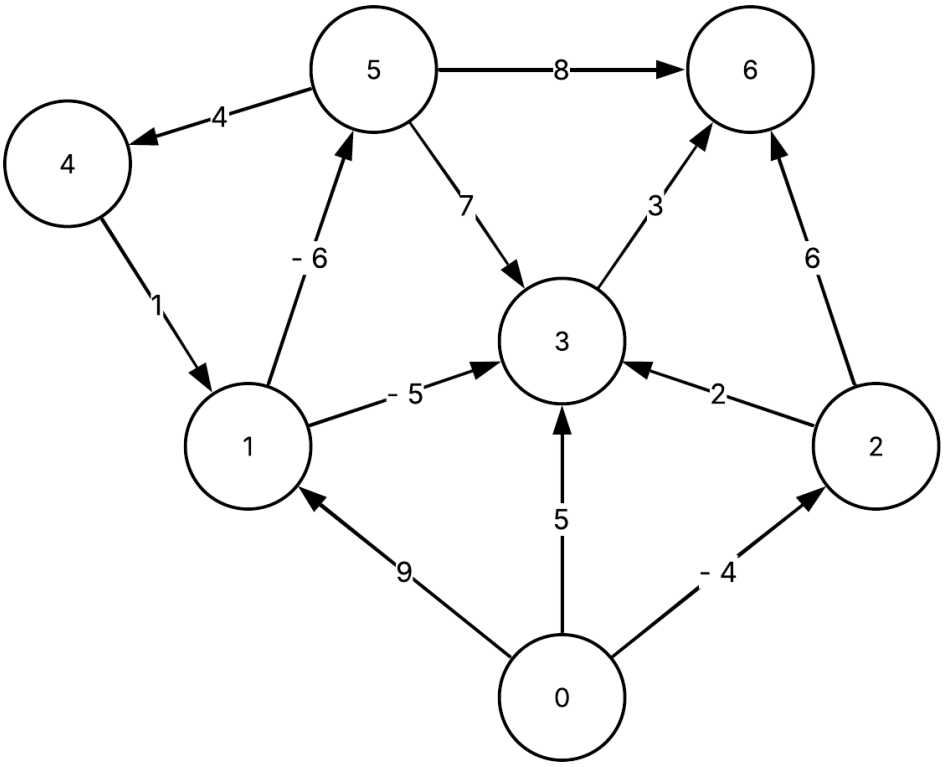


Figura 1. Grafo personalizado.

2.1 Primera iteración

Después de realizar la primera iteración sobre todos los vértices (asumiendo que estos se encuentran ordenados como se ejecutaría una BFS), se obtienen los siguientes valores como el mínimo para alcanzar cada nodo:

- Vértice 0: 0
- Vértice 1: 8
- Vértice 2: – 4
- Vértice 3: – 2
- Vértice 4: 7
- Vértice 5: 3
- Vértice 6: 1

Nótese que el vértice 1 tiene un valor de 8 (y no 9) porque se realiza una iteración sobre el ciclo negativo.

2.2 Segunda iteración

Después de la segunda iteración, los valores mínimos para alcanzar cada vértice son los siguientes:

- Vértice 0: 0
- Vértice 1: 7
- Vértice 2: – 4
- Vértice 3: – 2
- Vértice 4: 6
- Vértice 5: 2
- Vértice 6: 1

3. Análisis de complejidad

Para analizar la complejidad, se debe evaluar el ciclo anidado más denso del algoritmo, el cual se muestra a continuación.

```
C/C++
for (int i = 0; i < numeroVertices - 1; ++i)
    for (Edge arista : listaAristas)
      if (dist[arista.origen] != INF)
        if (dist[arista.origen] + arista.peso < dist[arista.destino])
          dist[arista.destino] = dist[arista.origen] + arista.peso;
```

Código 1. Único ciclo anidado del algoritmo Bellman Ford.

Como se observa, se hacen E iteraciones dentro de un ciclo que se repite V veces, por lo que la complejidad total es $O(V \cdot E)$. En grafos completos, donde la cantidad de aristas es

$$E = \frac{V(V-1)}{2},$$

el algoritmo adquiere una complejidad máxima de $O(V^3)$.

Para la segunda etapa del algoritmo (la cual tiene una complejidad menor), se observa el siguiente ciclo:

```C/C++
for (Edge e : listaAristas)
  if (dist[e.origen] + e.peso < dist[e.destino])
   return true;
```

Código 2. Único ciclo no anidado del algoritmo Bellman Ford.

En el peor de los casos, la complejidad de este segmento será $O(E)$ o $O(V^2)$ para grafos completos.